

— 면접용 요약

안세웅

분산 환경의 데이터 정합성과 장애 격리를 고민하는 백엔드 개발자

CORE COMPETENCIES

핵심역량

백엔드를 중심으로 설계하고, 필요하면 화면까지 직접 붙입니다. 재난·기상 실시간 알림 서비스 (SafeAlert)를 기획부터 배포까지 단독 수행하며 5개 마이크로서비스와 API Gateway를 설계 했고, O2O 플랫폼(LocalQuest)에서는 팀 리더로 Spring·React 풀스택을 담당했습니다.

p95 30% ↓

부하 테스트 기반 성능 개선

MSA 5

마이크로서비스 단독 설계

replica 3

WebSocket 무손실 (Redis Pub/Sub)

PROJECT 01

SafeAlert

재난·기상·미세먼지 공공 API를 구독하고 **WebSocket**으로 실시간 알림을 받는 MSA 기반
구독 서비스. 5개 마이크로서비스 + API Gateway를 단독 설계·구현·배포.



두 Kafka 토픽(alert.raw → alert.processed)이 수집·가공·알림 3단계를 서로 다른 속도로 독립 확장시킴 · 구독 등록은 Transactional Outbox로 이벤트 유실 방지



k6 부하 테스트 오류율 97.85% → 0%

오류율 97.85% → 0%

p95 응답 2.66s → 1.86s -30%



문제 상황

k6로 `POST /api/auth/login` 에 100 VU, 50초(10s 증가 → 30s 유지 → 10s 감소) 부하 테스트를 처음 돌렸을 때 오류율 97.85%로 거의 모든 요청이 실패했습니다.



원인 분석

원인이 두 겹이었습니다. ① API Gateway의 `RateLimitFilter` 가 IP당 분당 60건(`MAX_REQUESTS=60` , `WINDOW=1분`)으로 막아두는데, k6가 로컬 1개 IP에서 100명분 요청을 한꺼번에 쏘다 보니 정상 트래픽조차 대부분 429로 튕겨나갔습니다. 화이트리스트를 넣고 재실행하니 에러율은 0%로 떨어졌지만 p95 응답시간이 2.66s로 목표(2s)를 여전히 넘겼습니다 — ② `auth-service` 의 HikariCP `maximum-pool-size` 가 기본값 10이라, 100명이 동시에 로그인하면 bcrypt 해싱 검증이 끝날 때까지 90명이 커넥션 자리를 기다리며 지연되는 구조였습니다.



해결 과정

① `RateLimitFilter.java` 에 로컬 부하 테스트 전용 화이트리스트(127.0.0.1, 0:0:0:0:0:0:1)를 추가하고 ② `auth-service/application.yml` 의 `hikari.maximum-pool-size` 를 10 → 30으로 올린 뒤 재테스트했습니다.



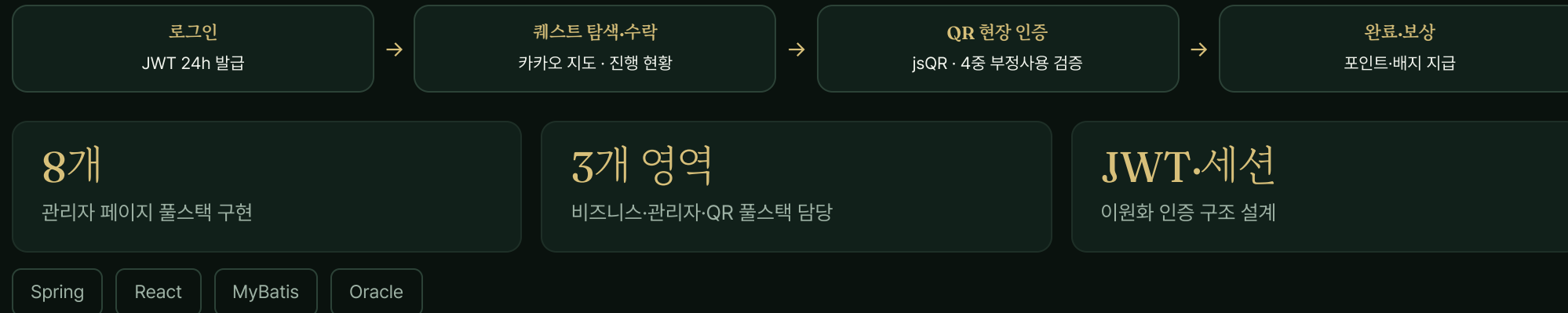
결과

1,475건 요청·p95 2.66s → 1,818건 요청·p95 1.86s로 목표(2s) 이내 달성. 평균 응답시간도 1.78s → 1.22s로 31% 줄었습니다. **배운 점:** 커넥션 풀 기본값(10)은 실무 동시 요청 규모를 가정한 값이 아닐 수 있다 — 부하 테스트로 실측하지 않으면 병목이 Rate Limit인지 DB 풀인지조차 알 수 없다.

PROJECT 02

LocalQuest

지역을 게임처럼 탐험하는 미션 기반 O2O 로컬 발견 플랫폼. 팀 5명 중 **팀 리더**로 QR 현장 인증 시스템을 설계·구현하고, 비즈니스 React SPA와 관리자 8개 페이지를 풀스택으로 담당.



관리자 인증 공백 → 전 경로 검증 + 이중화

관리자 경로 인증 일부 경로 검증 공백 → 전 경로 검증 + 이중화 



문제 상황

관리자 페이지는 `RoleBasedPageInterceptor` 가 세션의 `ADMIN` 역할을 검사하는데, 점검 중 인터셉터가 적용되지 않아 검증 없이 접근 가능한 관리자 경로가 있는 걸 발견했습니다.



원인 분석

`servlet-context.xml` 의 인터셉터 등록에 `<mapping path="/admin/**"/>` 만 있었습니다. 스프링 AntPathMatcher는 `/admin/**` 를 하위 경로에만 매칭시켜서, trailing 세그먼트가 없는 `/admin` 루트 자체(관리자 진입점, `AdminController` 가 매핑된 경로)는 매칭 대상에서 빠집니다. 인증을 인터셉터 한 곳에만 의존하다 보니 이 매핑 누락이 그대로 무방비 구간으로 남아 있었습니다.



해결 과정

`servlet-context.xml` 에 `<mapping path="/admin"/>` 을 `/admin/**` 와 나란히 추가해 루트 경로까지 커버하도록 정비했습니다. `RoleBasedPageInterceptor` 의 `resolveAllowedRoles()` 가 `requestPath.equals(routePrefix) || requestPath.startsWith(routePrefix + "/")` 로 정확 일치·하위 경로를 모두 검사하는 것도 다시 확인했고, 마스터 계정(userId=1) 보호 같은 핵심 분기는 컨트롤러 레벨에서도 한 번 더 검증하도록 이중화했습니다.



결과

`AdminController` · `AdminBusinessAuthController` · `AdminBusinessQrController` 등 관리자 컨트롤러가 모두 `/admin` 하위 경로라, 매핑 정비 후 모든 관리자 경로가 인증·역할 검증을 거칩니다. **배운 점:** 인터셉터 경로 패턴은 프레임워크의 매칭 규칙(`/**` 는 자기 자신을 포함하지 않는다는 것)까지 알아야 안전하다 — 적용 범위를 명시적으로 관리하고 핵심 로직은 다층 방어해야 한다.

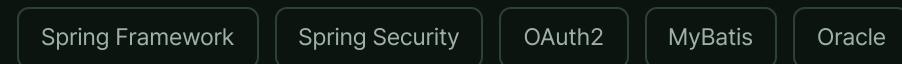
PROJECT 03

HashTrip

여행 성향 분석부터 일정 공유까지, 내 취향에 맞는 여행을 계획하는 플랫폼. 설문 **가중치 합산**
기반 **성향 분석 알고리즘**을 직접 설계·구현.



한국관광공사 공공API로 여행지 데이터 수집 · Google·Kakao·Naver OAuth2 소셜 로그인 3종 연동



중복 태그 오산출 → 가중치 합산 방식 전환

성향 분석 결과 중복 태그 사오산출 → 오류 케이스 0건 



문제 상황

1:1 선택지 설문 10문항에 답변을 마치면 최종 여행 스타일을 산출하는데, 초기 버전은 `TRAVEL_ANALYSIS_MASTER` 테이블에서 문항별(Q1_PLACE·Q2_PLAN·Q9_ENERGY) 선택 조합키를 그대로 조회해 매칭하는 방식이었습니다. 동일 태그가 여러 질문에 걸쳐 중복 선택되면 실제 가장 많이 고른 태그가 아닌 다른 태그가 최종 성향으로 결정되는 오류가 나왔습니다.



원인 분석

기존 `getFinalAnalysisResult` 조회는 선택한 조합키가 마스터 테이블에 존재하지만 보는 문자열 매칭이라, 같은 태그를 여러 문항에서 반복 선택해도 그 빈도가 결과에 반영되지 않았습니다. 선택 횟수를 세는 로직 자체가 없어서, 1번 선택된 태그가 매칭 순서상 먼저 걸리면 3번 선택된 태그를 제치고 그대로 최종 결과가 되는 구조였습니다.



해결 과정

`UserTagMapServiceImpl.processUserAnalysis()` 를 전면 수정했습니다. `TRAVEL_ANALYSIS_MASTER` 를 걷어내고 `TAG_WEIGHT_MASTER` · `TRAVEL_RESULT_MAPPING` 테이블을 새로 설계해, `users_mapper.xml` 의 `getTravelAnalysisData` 쿼리에서 태그 코드를 카테고리(PLACE/ENERGY/PLAN)로 매핑한 뒤 `SUM(W.WEIGHT) ... GROUP BY W.CATEGORY` 로 문항별 선택 빈도를 합산하도록 바꿨습니다. 질문 3개(QUESTION_ID 1, 2, 9) 각각에서 합산 점수가 가장 높은 카테고리를 뽑아 최종 문구를 구성합니다.



결과

중복 태그가 있는 모든 케이스에서 실제 선택 빈도가 가장 높은 태그가 최종 여행 스타일로 결정되도록 수정 후 오류 케이스 0건을 확인했습니다. 문자열 비교 → 가중치 계산으로 방식을 바꾼 덕분에, 이후 성향 문항이 늘어나도 카테고리·가중치 테이블만 추가하면 로직 변경 없이 확장할 수 있습니다.

RETROSPECTIVE

회고 및 배운 점



기술적 성장

이벤트 유실·다중 인스턴스 세션·인증 공백·알고리즘 엣지 케이스처럼 겉으로는 다른 문제들이 결국 같은 질문("코드가 실제로 그렇게 동작하는가")으로 수렴한다는 걸 세 프로젝트에서 반복해 확인했습니다.



역할에 따른 시야

SafeAlert는 기획부터 배포까지 단독 수행, LocalQuest는 팀 리더로 API-first 협업, HashTrip은 발표 담당으로 전체 아키텍처 통합 — 역할이 바뀔 때마다 요구되는 시야도 달라진다는 걸 체감했습니다.



다음 목표

LocalQuest의 모놀리식 한계를 넘고자 SafeAlert에서 MSA·Kafka·Kubernetes 기반 분산 시스템 신뢰성 설계(재시도·서킷 브레이커·관측성)를 직접 다졌고, 이 경험을 AI 하네스 엔지니어링으로 확장하려 합니다.

— *Thank you*

감사합니다

더 궁금한 부분은 아래 채널로 편하게 연락 주세요.

dpcks2553@naver.com

[GitHub ↗](#)

[기술 블로그 ↗](#)